MYELIN PASSIVE PERSISTENCE RECORDER - PHASE 2 DESIGN
26 September 2026 | Amended design: public Study category | No production implementation

1. DECISION AND BOUNDARIES

The bounded passive recorder is feasible within the four-file allowlist. One PERSISTENCE/secant_recorder.py component subscribes to existing public projections and submits immutable records through EngineStorage. EngineStorage facades already share the one serialized SQLite writer lane. No database.py change, second writer, recorder thread, organ command, classification algorithm, or sensing change is needed.

This document proposes the exact contract for review. It does not authorize or perform a migration. The earlier audit's inability to prove lossless collection is accepted as a limitation, not misrepresented as missing SECANT data. A successful flush proves only that this recorder's admitted records committed, never that it saw every organ event.

Data direction: public projections -> passive recorder -> EngineStorage._submit -> existing shared SQLite lane -> SQLite. The recorder may read immutable public values and storage results. It must not invoke authority methods, Controller commands, snapshot/drain methods on organs, or popup actions. Launcher may connect/disconnect signals and seed the existing current_session_id() read accessor.

Two new tables only: secant_events and study_resource_events. Recorder health is persisted as explicitly namespaced myelin_meta entries, not fake SECANT events or a third recorder table. Existing study_resources remains the sole durable PRIMARY/LATER authority.

Approved design amendment: add study_app_category TEXT NOT NULL and include it in the resource-state signature. The source is the existing public control_view['study_app_category']['category'] string, not the whole metadata object. Preserve the category verbatim when its evidence and identity are valid; otherwise store an empty string as unavailable. This is a design/DDL-test amendment only. The four-file production allowlist and stop-before-implementation boundary remain unchanged.

2. EXACT PROPOSED SCHEMA: SECANT_EVENTS

The following DDL is proposed, not installed. All identifiers are recorder-side unless explicitly described as native. SQLite signed 64-bit integer bounds and strict Python type validation apply before submission. Empty-string optional native fact fields remain empty strings; they must not be guessed.

```
CREATE TABLE secant_events (
    event_id TEXT NOT NULL PRIMARY KEY CHECK (length(event_id) = 64),
    contract_version INTEGER NOT NULL CHECK (contract_version = 1),
    session_id INTEGER NOT NULL REFERENCES sessions(id),
    study_context_id TEXT NOT NULL CHECK (length(study_context_id) > 0),
    study_target_resource_id TEXT NOT NULL,
    episode_id TEXT NOT NULL CHECK (length(episode_id) > 0),
    fact_digest TEXT NOT NULL CHECK (length(fact_digest) = 64),
    occurrence_ordinal INTEGER NOT NULL CHECK (
        typeof(occurrence_ordinal) = 'integer' AND occurrence_ordinal > 0),
    event_name TEXT NOT NULL CHECK (event_name IN (
        'SECANT_EPISODE_FOUNDED', 'SECANT_DISPLAYED',
        'SECANT_ACTION_INTENT', 'SECANT_FROZEN', 'SECANT_RESUMED',
        'SECANT_RETURN_STARTED', 'SECANT_RETURN_VERIFIED',
        'SECANT_RECOVERY_CLOSED', 'SECANT_RECOVERY_INTERRUPTED',
        'SECANT_EPISODE_CLOSED')),
    stage INTEGER NOT NULL CHECK (typeof(stage) = 'integer' AND stage BETWEEN 0 AND 3),
    occurred_monotonic_ns INTEGER NOT NULL CHECK (
        typeof(occurred_monotonic_ns) = 'integer' AND occurred_monotonic_ns >= 0),
    request_id TEXT NOT NULL,
    attempt_id TEXT NOT NULL,
    evidence_id TEXT NOT NULL,
    reason TEXT NOT NULL,
    canonical_fact_json TEXT NOT NULL CHECK (json_valid(canonical_fact_json)),
    recorder_run_id TEXT NOT NULL CHECK (length(recorder_run_id) > 0),
    recorder_received_monotonic_ns INTEGER NOT NULL CHECK (
        typeof(recorder_received_monotonic_ns) = 'integer'
        AND recorder_received_monotonic_ns >= 0),
    recorder_received_at_utc TEXT NOT NULL,
    recording_quality TEXT NOT NULL CHECK (recording_quality IN (
        'BOUNDED_OBSERVATION', 'RECORDING_GAP_DETECTED')),
```

```
    recorded_at_utc TEXT NOT NULL,
    UNIQUE (session_id, study_context_id, episode_id,
            fact_digest, occurrence_ordinal),
    CHECK (event_name != 'SECANT_DISPLAYED' OR stage BETWEEN 1 AND 3),
    CHECK (event_name != 'SECANT_ACTION_INTENT' OR reason = 'RETURN TO STUDY'),
    CHECK (event_name NOT IN ('SECANT_FROZEN', 'SECANT_RESUMED')
           OR reason = 'uncertainty')
);
CREATE INDEX ix_secant_events_session_episode_time
    ON secant_events(session_id, study_context_id, episode_id,
                     occurred_monotonic_ns, event_id);
```

TimelineFact.at_ns is stored exactly as occurred_monotonic_ns. There is deliberately no occurred_at_utc column. recorded_at_utc is generated inside the writer callback at insertion, not when the signal arrives, and is not the exact SQLite commit instant. Recorder receipt stamps are separate provenance.

Store the native names. SECANT_DISPLAYED retains stage 1/2/3. Recovery facts legitimately carry stage 0; do not borrow the projection's current stage. The action stream includes accepted SECANT_ACTION_INTENT with reason RETURN TO STUDY only. SECANT_ACTION_REJECTED and NEED BREAK are outside this requested stream. Hold facts are filtered to reason uncertainty. All closure reasons remain verbatim, without an enum that could discard a later native reason.

SECANT_RETURN_VERIFIED is also the existing transition into closure confirmation; do not insert a second invented CLOSURE_CONFIRMATION_STARTED fact. Friendly labels belong in a future reader. Popup closure does not imply episode closure.

3. EXACT PROPOSED SCHEMA: STUDY_RESOURCE_EVENTS

These are observed resource-state records. Their event kind does not assert an exact native RI transition. Block/generation may remain unknown, while session, context token and resource key are mandatory. Missing identity means defer within the bounded buffer or reject with an explicit gap, never guess.

```
CREATE TABLE study_resource_events (
    event_id TEXT NOT NULL PRIMARY KEY CHECK (length(event_id) = 64),
    contract_version INTEGER NOT NULL CHECK (contract_version = 1),
    session_id INTEGER NOT NULL REFERENCES sessions(id),
    recorder_run_id TEXT NOT NULL CHECK (length(recorder_run_id) > 0),
    observation_ordinal INTEGER NOT NULL CHECK (
        typeof(observation_ordinal) = 'integer' AND observation_ordinal > 0),
    event_kind TEXT NOT NULL CHECK (event_kind IN (
        'RESOURCE_STATE_FIRST_OBSERVED', 'RESOURCE_STATE_CHANGE_OBSERVED')),
    study_context_token TEXT NOT NULL CHECK (length(study_context_token) > 0),
    study_block_id TEXT,
    study_generation INTEGER,
    resource_key TEXT NOT NULL CHECK (length(resource_key) > 0),
    display_title TEXT NOT NULL,
    process_name TEXT NOT NULL,
    resource_surface TEXT NOT NULL,
    study_app_category TEXT NOT NULL,
    classification TEXT NOT NULL,
    measured_classification TEXT NOT NULL,
    effective_operational_state TEXT NOT NULL,
    confidence REAL NOT NULL CHECK (
        typeof(confidence) IN ('integer', 'real') AND confidence BETWEEN 0 AND 1),
    measured_confidence REAL NOT NULL CHECK (
        typeof(measured_confidence) IN ('integer', 'real')
        AND measured_confidence BETWEEN 0 AND 1),
    user_confirmation TEXT NOT NULL,
    state_digest TEXT NOT NULL CHECK (length(state_digest) = 64),
    canonical_state_json TEXT NOT NULL CHECK (json_valid(canonical_state_json)),
    source_lane TEXT NOT NULL CHECK (source_lane IN (
        'CONTROL_VIEW', 'RESOURCE_INTENT_LIVE_VIEW')),
    timestamp_basis TEXT NOT NULL CHECK (timestamp_basis IN (
        'COHERENT_FOREGROUND_OBSERVATION', 'RECORDER_RECEIPT')),
    source_observation_sequence INTEGER,
    source_observed_monotonic_ns INTEGER,
    source_observed_at_utc TEXT,
    recorder_received_monotonic_ns INTEGER NOT NULL CHECK (
        typeof(recorder_received_monotonic_ns) = 'integer'
        AND recorder_received_monotonic_ns >= 0),
    recorder_received_at_utc TEXT NOT NULL,
    provenance_json TEXT NOT NULL CHECK (json_valid(provenance_json)),
    recording_quality TEXT NOT NULL CHECK (recording_quality IN (
        'BOUNDED_OBSERVATION', 'RECORDING_GAP_DETECTED')),
    recorded_at_utc TEXT NOT NULL,
    UNIQUE (recorder_run_id, session_id, observation_ordinal),
    CHECK ((study_block_id IS NULL AND study_generation IS NULL) OR
           (study_block_id IS NOT NULL AND length(study_block_id) > 0
```

```
            AND study_generation IS NOT NULL
            AND typeof(study_generation) = 'integer' AND study_generation > 0)),
    CHECK (
        (timestamp_basis = 'RECORDER_RECEIPT'
         AND source_observation_sequence IS NULL
         AND source_observed_monotonic_ns IS NULL
         AND source_observed_at_utc IS NULL)
        OR
        (timestamp_basis = 'COHERENT_FOREGROUND_OBSERVATION'
         AND source_lane = 'CONTROL_VIEW'
         AND source_observation_sequence IS NOT NULL
         AND typeof(source_observation_sequence) = 'integer'
         AND source_observation_sequence > 0
         AND source_observed_monotonic_ns IS NOT NULL
         AND typeof(source_observed_monotonic_ns) = 'integer'
         AND source_observed_monotonic_ns >= 0
         AND source_observed_at_utc IS NOT NULL
         AND length(source_observed_at_utc) > 0)
    )
);
CREATE INDEX ix_study_resource_events_scope_order
    ON study_resource_events(session_id, study_context_token,
                             recorder_run_id, observation_ordinal);
CREATE INDEX ix_study_resource_events_resource
    ON study_resource_events(session_id, study_context_token, resource_key);
```

Storage validates canonical UTC strings, finite confidence values, enum values supported by contract v1, non-boolean integers, required identities, canonical JSON equivalence and IDs. SQL complements validation; json_valid alone does not prove a payload's shape. Unknown enums cause a visible unsupported-contract gap, not reinterpretation. Tables have NO CASCADE deletion, triggers, historical backfills or additional writer ownership.

The original normalized/truncated display_title is stored exactly. No confirmation_source, exact confirmation-response timestamp, inferred platform, visit ordinal, or invented full title is stored. resource_surface is not renamed platform. No occurred_at_utc or classification_changed_at field is introduced.

study_app_category stores the public metadata object's category member exactly, without trimming, case changes, aliases, classification or derivation from the title, process, surface, RI classification or confirmation. Do not store its label member or serialize the metadata object into this TEXT field. Accept the category only from the same identity-validated control_view and with evidence_available=true and a string category. The public unavailable result uses category='' and evidence_available=false; missing/unusable category evidence and standalone RI fallback also store ''. Do not carry a previous category forward. Malformed or mismatched metadata is retained as a rejection reason in provenance_json; it must not lend a category to another observation. Category availability does not change the separate RI identity/timestamp validation rules.

Future readers may present DISTRACTION / SOCIAL / confirmed when those separate public fields support it. An empty category is displayed as 'category unavailable', never guessed as SOCIAL or OFF TASK. 'confirmed' is based on user_confirmation, not on category. Any displayed time must retain the observation/receipt provenance already specified here.

PRIMARY/LATER is deliberately not duplicated in this table. A storage reader resolves role from study_resources using session_id + study_context_token + resource_key, additionally enforcing block/generation when present. Exactly one compatible durable resource identity is required; zero matches returns PENDING_OR_UNQUALIFIED, multiple matches returns AMBIGUOUS. It returns resource_fact_id, authoritative block/generation, identity kind/version, role and qualification times separately. If no block/generation was initially available, the unique durable row may supply it to the read result, never rewrite history.

A qualified role may become readable later without generating an RI transition. Readers must label it as authoritative qualification metadata, including qualified_at_utc; they must not claim the role already existed at the earlier observation. PRIMARY means primary qualified resource, not first visit. Neither runtime pending_resources nor an uncommitted writer receipt establishes a role.

Reader role labels are PRIMARY RESOURCE and LATER RESOURCE. They must not claim '1st resource', '2nd resource' or first-ever/second-ever visit. observation_ordinal below is recorder ordering only, not an authoritative visit ordinal.

4. RECORDER RESPONSIBILITIES AND IDENTITY BINDING

The proposed module contains one QObject-based PassiveSecantRecorder, its immutable persistence-input dataclasses and pure canonicalization helpers. Storage accepts these contracts via a type-only dependency or local import; avoid import-time cycles. Reading immutable public types is permitted; invoking organ authorities is not.

Public recorder API:
attach(controller, initial_session_id) -> None
on_secant_projection(value) -> None
on_resource_intent_live_view(value) -> None
on_control_view(value) -> None
on_study_view(value) -> None
on_session_changed(session_id) -> None
on_lifecycle(kind, payload=None) -> None
quality_snapshot() -> immutable RecorderQuality
begin_shutdown() -> None
finish_shutdown(timeout_seconds=8.0) -> FlushResult

Its responsibilities are validation, provenance capture, source binding, native-event selection, deduplication, bounded buffering, asynchronous submission, receipt reconciliation and health reporting. It owns no domain timer, dwell, episode decision, return verification, classification or Study-resource qualification algorithm. QTimer is only a scheduling mechanism for persistence work and receipt checks.

Session binding must precede persistence. SECANT StudyScope.context_id must agree with the current session's exact prefix str(session_id) + ':' and a nonempty token; retain the entire native context_id and target_resource_id. Process completed episodes with each episode's own scope, even when the current projection.scope is None. The fact episode_id must match its enclosing episode. Process old completed scopes within the same session without reassigning them to the current Study block.

StudySnapshot is distinguished from CosecantSnapshot on the shared study_view_changed signal. Use the public block_id/generation/context_token values for exact context mapping, with source-session provenance. Do not reproduce Study's hash or qualification algorithm. A coherent control_view has its own session_id and can validate a Study/RI token association. Bare RI values without a session envelope require an already-established current-session context association; otherwise do not commit them.

On session None, retire the active binding and seal the old session's buffered records with their captured session IDs. Do not clear pending receipts or relabel records. A new session creates new binding/RI ordering state. A reset is not SECANT_EPISODE_CLOSED. A late payload with a retired/mismatched session or context is rejected and reflected in quality. Unknown association is not silently attached to whichever session happens to be current.

5. SECANT DEDUPLICATION: MULTIPLICITY, NOT JUST HASHING

A fingerprint alone would collapse two facts whose immutable fields are identical. Therefore contract v1 uses canonical content plus an occurrence rank within that content's cumulative episode timeline. This rank is recorder-derived evidence position, not a native event sequence.

Canonical JSON is a versioned fixed-shape array serialized as UTF-8 with no Unicode normalization, no whitespace, fixed field order, ensure_ascii=False and allow_nan=False. Integers remain integers, optional native strings remain exact strings.
Fields, in order:
[1, session_id, scope.context_id, scope.target_resource_id, episode_id,
fact.name, int(fact.stage), fact.at_ns, fact.request_id,
fact.attempt_id, fact.evidence_id, fact.reason]

fact_digest = SHA-256(canonical UTF-8 bytes), lower-case hex.
occurrence_ordinal = 1-based rank of this exact canonical fact among identical facts in this episode's authoritative cumulative tuple.
event_id = SHA-256(canonical JSON ["secant-event-v1", canonical_fact_array, occurrence_ordinal]).

For each (session, scope, episode), reconcile current timeline and completed[].timeline before enumerating; choose the longer only if they are identical/prefix-compatible. Do not concatenate them and double-count overlapping facts. Enumerate the original timeline without inventing facts. Reject mismatched episode entries. Ignore pre-episode facts with empty episode_id for this stream. Require consistent target identity across appearances.

Example: [F] -> [F,F] produces F#1 then F#2. Replaying [F,F] inserts nothing. Repeating the same display title at a later at_ns or another request_id is already a different F. Current+completed copies of the same tuple do not add multiplicity.

Within a running attachment, verify observed timeline prefixes, not only lengths. Regressions, altered prefixes, inconsistent completed copies and unknown truncation enter RECORDING_GAP_DETECTED; do not renumber ambiguous suffixes. Persist only already-unambiguous facts. A fresh attachment may accept the full presented cumulative tuple, but is always bounded: it cannot detect an unseen earlier truncation or a truly distinct identical event never coexisting in a received tuple.

The DB unique key and event_id exclude recorder_run_id, making an unchanged cumulative timeline idempotent across recorder attachments. A duplicate key requires exact semantic equality, including canonical content and ordinal, before it is treated as a retry. Existing first-capture provenance and recorded_at_utc are preserved. A hash collision with different canonical bytes is a hard recorder conflict, not ON CONFLICT DO NOTHING. SHA-256 plus full-payload comparison is collision-detecting, not a mathematical promise of collision impossibility.

Do not mark a candidate committed on receipt RUNNING. Maintain separate pending/in-flight/committed state. Cache eviction may cause safe DB retries but must never reset the canonical occurrence calculation. If validation history cannot be retained within limits, flag coverage degradation rather than pretending the next partial projection is a complete new timeline.

6. RESOURCE TRANSITIONS AND TIMESTAMP LAW

Meaningful state signature, in fixed order:
[1, session_id, study_context_token, resource_key, display_title,
process_name, resource_surface, study_app_category, classification, measured_classification,
effective_operational_state, confidence, measured_confidence,
user_confirmation]

First accepted signature for a context is RESOURCE_STATE_FIRST_OBSERVED. A different signature from the most recently admitted state of that context is RESOURCE_STATE_CHANGE_OBSERVED. Compare the current resource, not only a per-resource last-value map: A -> B -> A must produce three observations. Same state on repeated projections produces no additional row. Exact finite confidence changes count as changes; no unapproved rounding or classification hysteresis is introduced. Saturation is handled as a gap, not by secretly erasing changes.

The category is part of canonical_state_json and state_digest. A category-only change on a later valid observation produces a new observed-state row even when resource_key and all RI fields are unchanged. Availability changes SOCIAL -> '' and '' -> SOCIAL also change the signature; repeated '' does not. An RI-only fallback with category='' followed by a coherent composite with category=SOCIAL is therefore a legitimate newly observed state, not a duplicate to suppress. This records a change in available observed evidence, not proof of the exact moment a native category changed. Existing sequence/conflict checks still apply. Contract_version remains 1 because this design has not been implemented or migrated; no deployed v1 rows are being reinterpreted.

state_digest is SHA-256 over canonical_state_json. observation_ordinal is a monotonically increasing recorder counter per (recorder_run_id, session_id), allocated once at admission. event_id hashes ["resource-observation-v1", recorder_run_id, session_id, observation_ordinal]. Retrying uses the original ID/ordinal/receipt stamps. A later A after B gets a new ordinal even if state_digest repeats. A new attachment begins a new run ID and a clearly labeled baseline observation; it cannot deduplicate an unobserved real transition across restart. Do not add a UNIQUE(state_digest) constraint.

The composite control_view is the preferred ordered resource source. It contains the foreground observation and RI view together. Require exact agreement among envelope session_id, metadata session_id, source ResourceObservation, foreground sequence/monotonic time/PID/HWND/raw title, canonical process, metadata resource key/context token and RI key/context token. Require nonempty producer observation token, supported metadata schema, and a non-null producer-supplied source_observation. Validate the fields directly; do not call build_observation(), observation_token() or any organ method. Do not reject solely because the separate display-category evidence_available is false: category classification is not the RI coherence contract.

For coherent records, copy foreground.observed_monotonic_ns and observed_wall_time_utc exactly into source_observed_* and use COHERENT_FOREGROUND_OBSERVATION. Validate UTC format and ordering/identity; never compute UTC by offsetting a SECANT monotonic time. source_observation_sequence is ordering evidence, not proof that every intervening sample was delivered.

If source time or coherence is unavailable but current session/context identity remains established, use RECORDER_RECEIPT, null all source time/sequence columns, and preserve an explicit rejection reason in provenance_json. If session/resource/context itself is untrusted, do not write a state row. Recorder receipt time dates the recorder's observation only. Even coherent foreground time is not an exact classification-change or confirmation-response timestamp.

Cross-lane arbitration is deterministic. Before the first accepted control_view of a session, a session-bound RI live value may supply a receipt-timed baseline/transition. Once a valid session-bound composite resource value is accepted, control_view exclusively drives resource transitions until the session changes; standalone RI signals become validation/availability hints, not a second interleaved timeline. If the first composite matches the last fallback signature, suppress the duplicate and do not retroactively replace its receipt timestamp. This avoids creating a false A -> B -> A from delayed independent live projections. A composite with invalid timing may still supply a receipt-timed state when identity is valid. A disappeared composite lane does not silently promote the unsequenced lane; quality reports degraded coverage.

Within one session, reject a coherent sequence that goes backwards; equal sequence and unequal state is a conflict, not a newly observed change. Multiple skipped sequence numbers are not by themselves a proven gap because upstream coalescing is expected. New sessions reset sequence tracking. provenance_json retains metadata identity fields, available source sequence/token, source-rejection reason, attachment/binding basis and timestamp semantics. It contains no inferred confirmation_source.

7. EXACT ENGINESTORAGE ADDITIONS

New methods in PERSISTENCE/storage.py:

append_secant_events(events: tuple[SecantEventRecord, ...]) -> TransactionReceipt
append_study_resource_events(events: tuple[StudyResourceEventRecord, ...]) -> TransactionReceipt
record_passive_recorder_status(status: RecorderStatusRecord) -> TransactionReceipt
flush_passive_recorder() -> TransactionReceipt
read_passive_recorder_status(recorder_run_id: str) -> RecorderStatusRecord | None
read_study_resource_event_roles(event_ids: tuple[str, ...]) -> tuple[RoleResolution, ...]

Append methods validate a bounded immutable batch and call _submit once; the callback inserts the entire batch atomically. It checks duplicate identity against all semantic fields, preserving first-insert timestamps/provenance on exact retry. Conflicting IDs/payloads cause batch rollback and a FAILED receipt. Each callback returns existing TransactionResult; no new writer result transport is needed. recorded_at_utc is created in the callback. Existing receipt(), wait(), forget_receipt() and shutdown() are reused unchanged.

record_passive_recorder_status writes only its reserved myelin_meta key through _submit. It checks increasing revision numbers and immutable run identity. flush_passive_recorder submits a no-op TransactionResult callback as a FIFO barrier. Barrier COMMITTED means earlier accepted lane callbacks reached terminal states, not that they all succeeded; the recorder must independently verify its own receipts.

Readers use the existing read-only connection path, never bootstrap/migrate or acquire a writer for reads. Role resolution uses the exact matching rules in section 3 and returns AMBIGUOUS rather than multiplying rows. No role read occurs synchronously inside a Qt signal handler. Runtime recording does not require a role query: this association is safely deferred to the storage reader/report layer.

Signal handlers do no SQL and no waits. They retain immutable projection references and capture receipt/binding provenance. A recorder-owned GUI QTimer processes bounded units, constructs records and submits through EngineStorage; the only DB thread is the existing lane. This design does not add a worker thread or a second writable connection.

Proposed technical limits for review (not physiological timing): process at most 64 new records or 2 ms of extraction work per timer turn; at most one recorder write transaction in flight; each batch <=64 rows and <=256 KiB serialized; pending record memory <=2 MiB and <=2,048 rows; at most 8 retained projection references; a single accepted projection <=50,000 fact entries. Scan large cumulative histories incrementally; do not hash the entire history in a signal callback. Never replace a distinct admitted RI transition to meet limits. A new reference that exceeds capacity is refused with loss counters and gap status. A session switch retains old labeled pending work subject to the same caps.

All proposal limits are persistence admission controls only. One in-flight callback and small transactions limit pressure on the shared writer. They cannot guarantee zero latency impact on a physically shared GUI thread/disk/lane; no such hard real-time isolation claim is made.

8. QUALITY, HEALTH DURABILITY AND FAILURE CONTAINMENT

Do not use COMPLETE_OBSERVED_STREAM: it invites a stronger guarantee than this source proves. Coverage has exactly two values: BOUNDED_OBSERVATION initially, and sticky RECORDING_GAP_DETECTED after an identified loss/ambiguity for that recorder run. A clean flush never promotes coverage above BOUNDED_OBSERVATION or clears a prior gap.

Separate writer state: STARTING, ACTIVE, DRAINING, CLOSED, FAILED. These are recorder states, not organ states. An immutable quality_snapshot and recorder-owned quality_changed signal expose them without modifying Controller or application.py. Launcher subscribes only to a concise diagnostic sink; no dialog, intervention, or existing organ signal is emitted. No UI change is proposed.

Durable key: myelin_meta.key = 'passive_recorder:v1:run:' + recorder_run_id. Value is a versioned JSON object with run_id, contract_version, status_revision, writer_state, coverage, opened_at_utc, last_recorded_at_utc, closed_at_utc (nullable), attached_session_ids, last committed resource ordinal per session, committed row counters, pending/dropped/rejected counters, gap reasons, and final_flush_committed. These are recorder observations; no occurrence times are inferred. Initial

STARTING/ACTIVE metadata must commit before stream rows are submitted. Failure to commit it disables recording, not the app.

Per-row recording_quality captures capture-time coverage only. A reader combines it with final run quality; it must not infer that earlier rows imply a gap-free run when a later failure occurred. Missing run metadata means UNKNOWN recorder health; an open run left by abrupt exit means unclean/incomplete recorder termination, not proof of particular lost organ events. Do not rewrite a crashed run as successfully closed on restart.

Malformed payload, identity conflict, capacity loss, changed timeline prefix, serialization error, failed/missing receipt or deadline expiry latches an explicit reason. Unknown timestamps alone use receipt provenance and bounded quality; they are not automatically proven dropped events. Loss counts are labeled minimum known losses when exact cardinality is unavailable.

No exception escapes a recorder signal slot or timer callback. A failed batch is not declared committed. Proposed queue-full policy: retain the identical immutable batch for at most one delayed resubmission after 250 ms; a second failure disables further recorder admissions, preserves failure state and reports the gap. No busy retry loop. Other write/constraint failures disable recording immediately. RUNNING beyond the recorder's deadline is outcome-unknown: keep its receipt, do not resubmit as if it failed. Never forget a RUNNING receipt.

If SQLite is unavailable, persisting a failure marker may also fail. Surface the in-memory status and bounded stderr diagnostic, leave durable run open/unclean, and make no false promise that the gap marker reached disk. Do not close the app, stop Study, pause SECANT, reset authorities, change global DB settings or force another owner's lane shutdown. A shared database/disk failure can independently affect other users of that database; the recorder cannot promise hardware-level fault isolation.

9. LAUNCHER SUBSCRIPTIONS AND LIFETIME

The proven seam is CONTROL_STATION/launcher.py startup_completed(exit_code, window), with window.engine created by application.py. Retain recorder and its OWN EngineStorage facade in launcher's lifetime dictionary, with QObject parent=app rather than the window. Do not borrow window.storage, because application shutdown releases it before Qt exit. The additional facade shares the existing lane and keeps it alive through recorder finalization.

At a successful window handoff, construct the facade and recorder, connect signals, then seed current_session_id() using the public read accessor. No event-loop yield occurs between connections and seeding. Attachment might still occur after a session already began; flag late attachment as bounded and do not claim historical coverage. Constructor/attachment errors are contained and logged; release any successfully acquired recorder facade. No recorder is created when window is None.

Connect exactly:
secant_projection_changed -> on_secant_projection
resource_intent_live_status_changed -> on_resource_intent_live_view
control_view_changed -> on_control_view
study_view_changed -> on_study_view
current_session_id_changed -> on_session_changed
engine_started / engine_paused / engine_resumed / engine_stopped
engine_failed / engine_status_changed -> on_lifecycle wrappers

Pause/resume/status signals delimit recorder provenance only. None creates a SECANT freeze, recovery, closure or RI transition. All subscriptions are one-way reads. Store the connected slots so detach removes only this recorder's connections.

Connect app.aboutToQuit to begin_shutdown (no blocking wait). Wrap app.exec() with a try/finally in launcher; after the event loop returns, invoke finish_shutdown, then release the recorder's facade, then clear lifetime references. Use idempotent cleanup for failed startup or repeated notifications. This is composition/lifetime wiring only; no Controller/Application change.

10. SHUTDOWN AND FLUSH CONTRACT

On engine_stopped or current_session_id_changed(None), seal that session's admissions while allowing asynchronous persistence of already-captured rows; keep the recorder facade alive for the next session. Preserve final projections delivered during Controller's existing drain before None. Do not call Controller._drain_telemetry or shutdown from the recorder.

Application's existing shutdown calls engine.shutdown then window.storage.shutdown before Qt quit. The recorder is still subscribed during those public emissions. aboutToQuit detaches recorder inputs, stops its timer, records a shutdown cutoff, and marks DRAINING. No claim is made to recover Controller items not publicly emitted by that cutoff.

After app.exec returns, finish_shutdown pumps only recorder-owned pending extraction/submission/receipt work, without Qt event re-entry or organ calls. Use one overall proposed 8-second recorder drain deadline. Await all recorder batches, inspect

their COMMITTED/FAILED receipts, submit and confirm a FIFO flush barrier, then submit and confirm final CLOSED metadata with final_flush_committed=true. If the deadline expires or any batch fails, return INCOMPLETE/OUTCOME_UNKNOWN as appropriate and never write successful CLOSED metadata. A best-effort failure status may be attempted only within the remaining budget.

Forget only this recorder's terminal receipts after their results are consumed. Keep any unknown/RUNNING receipt observable until release; a timed-out transaction may still commit. Release the recorder facade in finally using existing EngineStorage.shutdown. The shared lane closes only if it is the last owner. Existing final-owner shutdown has its own bounded 8-second wait; worst-case added exit wait is therefore approximately 16 seconds plus scheduling overhead, NOT falsely advertised as a single 8-second bound. Shortening that existing bound would require a different contract and is not proposed.

The recorder cannot guarantee flush after crash, forced kill, power loss, or an engine that still has undelivered public output. Incomplete flush is a recording-quality failure, never permission to block/alter an organ's operation. No private lifecycle drain or new exit veto is introduced.

11. V7 -> V8 MIGRATION PLAN

Only schema.py owns installation. Raise SCHEMA_VERSION to 8, add the two table names to MANAGED_TABLES, add the proposed DDL and exact verifier, and adjust ensure_production_schema version dispatch. storage.py bootstrap remains the one process-start gateway before any shared writer owner exists. No migration is triggered by a signal, recorder constructor beyond existing bootstrap semantics, or reader.

Preconditions for future execution: reviewed design/implementation, stopped application and no writer process, consistent SQLite backup including any WAL state, verified backup integrity and recorded baseline counts/hashes for all existing tables. Prefer SQLite's backup facility on a stable connection; do not copy only a live WAL-mode .db file. No backup or migration of the current DB is performed in this design task.

For an existing v7 database: verify the v7 baseline contract without modifying it; begin one IMMEDIATE transaction; create only secant_events, study_resource_events and the three listed indexes; verify full column definitions, NOT NULL/CHECK/FK/unique/index contracts and absence of interfering triggers; run foreign_key_check/quick_check; update myelin_meta.schema_version to '8' LAST; commit. Both new tables initially empty; no historical SECANT/RI backfill. Use explicit statements within one transaction, or a single BEGIN-containing executescript, never an executescript that accidentally commits an existing transaction halfway through.

If either table/index name already exists in v7, fail visibly unless it is an explicitly reviewed resumable exact contract. Default is fail, not IF NOT EXISTS masking a mismatch. Any failure rolls back DDL and version change together. Preserve every existing row and schema object.

For existing v8: verify, do not silently recreate missing tables or advance metadata. Reject any version >8. The current code groups all versions >=6 into the v7 path; it must be split so v7/v8 are handled correctly. Preserve existing v6 -> v7 behavior as the first additive step before v8, and ensure fresh/older supported initialization creates the v8 tables before the final MANAGED_TABLES check. Do not rerun historical physiology/data conversions against v7. These changes stay in schema.py.

Future migration validation compares complete old-table contents and sqlite_master definitions, not only row counts, and checks that no triggers or unwanted indexes appeared. UTC validation remains in storage; SQL type/shape constraints do not certify domain correctness.

12. ROLLBACK PROCEDURE

A recorder-only disable on v8 should detach it while retaining the v8-compatible application and data. Reverting to the old v7 code against schema v8 is NOT valid: current schema.py explicitly refuses a newer version. Neither setting schema_version=7 alone nor deleting only the new source file is an acceptable rollback.

For a future full code/schema rollback, stop all writers first and take a verified backup of the CURRENT v8 database. Export/archive the two new streams and reserved recorder metadata before removal. Prefer an offline inverse migration that preserves all post-upgrade changes to the pre-existing tables. After asserting exact v8 schema, no unexpected dependencies/triggers, and the archived evidence:

```
BEGIN IMMEDIATE;
DROP TABLE study_resource_events;
DROP TABLE secant_events;
DELETE FROM myelin_meta
 WHERE substr(key, 1, length('passive_recorder:v1:run:'))
       = 'passive_recorder:v1:run:';
UPDATE myelin_meta SET value = '7'
```

```
 WHERE key = 'schema_version' AND value = '8';
-- The offline runner must assert exactly one version row changed,
-- verify the unchanged v7 tables/contracts and clean FK/integrity checks,
-- then COMMIT; otherwise ROLLBACK.
```

Associated indexes disappear with their new tables. Restore the three edited files to their pre-implementation copies and remove the new recorder module only after the schema rollback succeeds. This is a proposed destructive procedure for separately approved future rollback, not an action executed now. No database.py edits or permanent rollback helper in production are required.

Alternative: restore the pre-upgrade backup only when explicitly accepting loss of ALL later data. A backup restore must be done with every connection closed and a consistent WAL/SHM replacement procedure; never mix old main DB bytes with newer journal sidecars. Retain both backups. Crash or pre-commit migration failure normally requires rollback/reopen, not manual version downgrading.

13. EXACT IMPLEMENTATION ALLOWLIST

1) PERSISTENCE/schema.py
Version 8, two additive tables (including study_app_category TEXT NOT NULL), three indexes, verifier/version dispatch.
2) PERSISTENCE/storage.py
Six methods listed in section 7; immutable validation and shared-lane writes.
3) PERSISTENCE/secant_recorder.py (NEW)
One passive observer, DTOs, canonicalization, bounded admission, receipt/quality logic.
4) CONTROL_STATION/launcher.py
Composition, subscriptions, recorder/facade ownership and exit finalization.

No other production file. No proposed modifications to SECANT, Study, Resource Intent, Monitor, Controller, Application, COSECANT, SINE, MIRL, MINE, COSINE, browser sensing or PERSISTENCE/database.py. Tests and design artifacts remain outside production source. If implementation reveals a need for a fifth production file, stop and report the specific blocked contract; do not silently widen this allowlist.

14. REQUIRED IMPLEMENTATION ACCEPTANCE CHECKS (NOT RUN NOW)

Category-specific acceptance: exact public category storage; unavailable/missing evidence -> ''; NOT NULL rejection; no category derivation or stale carry-forward; category-only state changes; availability changes; repeated-category dedup; RI fallback -> composite category transition; mismatched metadata never supplying category; PRIMARY RESOURCE/LATER RESOURCE remaining qualification labels. The SQL checks below run on in-memory fixtures only; recorder behavior still requires future implementation tests.

Use disposable databases and synthetic immutable public projections outside production. Verify: repeat/current+completed dedup; identical-fact multiplicity; replay across attachments; altered-prefix rejection; out-of-order sessions; final completed projection with scope=None; stage-zero recovery facts; accepted versus rejected intent; exact native reason preservation; RI A/B/A transitions; identical state across lanes; backward/equal sequence conflicts; coherence mismatch and receipt-only fallback; absent title/role/provenance staying absent; role appearing only after durable commit; ambiguous role join; NaN/bool/integer-bound validation; collision conflict rollback; queue-full and disk/constraint failures; RUNNING versus COMMITTED handling; terminal receipt cleanup; one shared lane across Monitor/window/recorder; facade ownership on shutdown; flush deadline and abrupt run termination; migration atomicity/idempotence and preservation of old tables; inverse rollback preservation of old-table changes.

Assert zero Controller command/organ authority calls from the recorder; no SQLite connection creation in secant_recorder.py; no extra writer thread; no blocking wait in signal callbacks. Measure bounded GUI work/shared-writer pressure. Windows subscription/shutdown acceptance remains necessary after implementation. Successful SQL syntax verification of a design is not a behavioral test.

15. EVIDENCE AND WORK PERFORMED

Source inspected read-only in MYELIN2.0_ULTIMATE_TRIAL:
PERSISTENCE/database.py:142-307 (nonblocking submit, receipts, shutdown FIFO).
PERSISTENCE/storage.py:451-674 (shared facade ownership and bootstrap), 924-940 (receipt APIs), 1251-1271 (immutable Study resource write), 1799-1814 (owner release).
PERSISTENCE/schema.py:11-34,651-744 (version, managed tables, verification/dispatch).
CONTROL_STATION/launcher.py:86-124 (startup handoff and Qt lifetime).
CONTROL_STATION/application.py:14902,16449-16506 (engine and shutdown ordering).
CONTROL_STATION/startup.py:227-259 (window callback seam).
CONTROL_STATION/controller.py:215-254,1109-1215,1258-1265,1347-1395 (public lanes and terminal drain).
CORE_ENGINE/monitor.py:1036-1131,2441-2468,5246-5265 (public provenance producer/transport).
CORE_ENGINE/monitor.py:1106-1205 (amendment: category member, evidence_available and empty unavailable result; read-only).
CORE_ENGINE/resource_intent.py:520-550 (live fields; prior audit).
SECANT/read_model.py:32-113 and COSECANT_STUDY_MODE/study.py:921-983 (immutable contract).

No source or production database mutation was performed. Only this design document, its PDF and scratch document/DDL-validation artifacts are created. Proposed DDL is checked against SQLite in memory; that check does not run production code, migrate a file, or create a prototype recorder. Source/database preservation is hash-checked against this task's baseline at delivery. The earlier audit remains a point-in-time report; this design does not claim to have rerun the complete database audit.

Amendment verification: in-memory DDL checks cover the required TEXT column, exact category round-trip, empty-string round-trip, NULL rejection and category-bearing canonical payloads. A document-contract check verifies that the canonical signature includes study_app_category. These checks do not claim that an unimplemented recorder already performs category extraction or deduplication. The TXT and PDF contain the same amended design. No production implementation has begun.

PERSISTENCE RECORDER DESIGN READY FOR CHIEF REVIEW